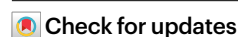


Efficient scaling of large language models with mixture of experts and 3D analog in-memory computing

Received: 27 May 2024

Accepted: 27 November 2024

Published online: 8 January 2025



Julian Büchel¹✉, Athanasios Vasilopoulos¹, William Andrew Simon¹, Irem Boybat¹, HsinYu Tsai², Geoffrey W. Burr², Hernan Castro³, Bill Filipiak⁴, Manuel Le Gallo¹, Abbas Rahimi¹, Vijay Narayanan⁵ & Abu Sebastian¹✉

Large language models (LLMs), with their remarkable generative capacities, have greatly impacted a range of fields, but they face scalability challenges due to their large parameter counts, which result in high costs for training and inference. The trend of increasing model sizes is exacerbating these challenges, particularly in terms of memory footprint, latency and energy consumption. Here we explore the deployment of ‘mixture of experts’ (MoEs) networks—networks that use conditional computing to keep computational demands low despite having many parameters—on three-dimensional (3D) non-volatile memory (NVM)-based analog in-memory computing (AIMC) hardware. When combined with the MoE architecture, this hardware, utilizing stacked NVM devices arranged in a crossbar array, offers a solution to the parameter-fetching bottleneck typical in traditional models deployed on conventional von-Neumann-based architectures. By simulating the deployment of MoEs on an abstract 3D AIMC system, we demonstrate that, due to their conditional compute mechanism, MoEs are inherently better suited to this hardware than conventional, dense model architectures. Our findings suggest that MoEs, in conjunction with emerging 3D NVM-based AIMC, can substantially reduce the inference costs of state-of-the-art LLMs, making them more accessible and energy-efficient.

Transformer-based large language models (LLMs) have demonstrated state-of-the-art capabilities across a spectrum of tasks^{1–4}, and their remarkable generative capacity has led to a transformative impact on the world. These advances are mostly a product of scaling laws⁵: a smooth increase in the performance of LLMs is observed by increasing the size of the model, assuming that the model is trained on sufficiently large amounts of data⁶. However, this has brought about a reality where state-of-the-art models easily surpass the billion-parameter threshold, some of them even exceeding hundreds of billions⁷.

A crucial step towards enabling the widespread adoption of this technology across various services and products involves reducing the hardware cost and the inference cost in terms of time and energy. The ongoing trend of ever-increasing model sizes, however, contradicts that objective, because the inflated parameter count incurs an increase in memory footprint, memory transactions and compute latency. Moreover, the primary hardware elements used for serving such models, notably graphics processing units (GPUs), are power-intensive and not designed for LLM inference. Given this, a key concern is reducing

¹IBM Research Europe, Rüschlikon, Switzerland. ²IBM Research – Almaden, San Jose, CA, USA. ³Micron Technology, Folsom, CA, USA. ⁴Micron Technology, Novi, MI, USA. ⁵IBM Thomas J. Watson Research Center, Yorktown Heights, NY, USA. ✉e-mail: jub@zurich.ibm.com; ase@zurich.ibm.com

the energy requirements of chips in LLM inference scenarios. This is crucial as the high power consumption and the associated costs often present limitations in a variety of use-cases.

A promising approach^{8,9} to alleviate the inference cost associated with LLMs involves embracing conditional computing, which is a concept where the trajectory of an input through a neural network is dynamically determined, and only a fraction of the parameters is utilized to process the input. These disjoint sets of parameters are referred to as ‘experts’, and models based on this concept are often called a ‘mixture of experts’ (MoE). The first promising implementation building on this idea, the switch transformer, was proposed in refs. 10,11. This architecture replaces every other feedforward network (FFN) in the transformer architecture with E expert FFNs. During the forward pass, a router network dynamically selects the top- k expert for processing the incoming activations at each modified FFN layer. The outputs from the selected experts are subsequently aggregated with a weighted sum (Fig. 1a).

Although earlier works have indicated that MoEs can surpass conventional dense models in performance, with lower inference cost^{12,13} and with more resource-efficient training¹³, MoEs typically required up to an order of magnitude more parameters just to match the performance of their dense, floating-point operations (FLOPs)-equivalent counterparts^{14,15}. For this and other reasons such as training instability, MoEs were often dismissed as an interesting approach to scaling models, though impractical for deployment due to their size, which introduced the need for model parallelism at the expert level, incurring additional communication overhead¹¹.

The assumption that MoEs require more parameters was refuted with the publication of the Mixtral network¹, a switch-like MoE that demonstrates the best cost/performance ratio among contemporary LLMs. It surpasses LLaMA 2² and GPT3.5⁴ on numerous benchmarks, exhibiting faster inference times—six times faster than LLaMA 2—and fewer parameters (47 B compared to LLaMA’s 70 B). Additionally, a study by Csordás, Irie and Schmidhuber¹⁶ revealed that with further optimizations, MoEs are able to consistently outperform their dense counterparts in parameter-equivalent scenarios while achieving speed-ups of up to 8×. To summarize, although MoEs were once deemed impractical, recent advancements demonstrate that MoEs can not only outperform dense models, but they also offer substantially cheaper inference, with faster processing times and fewer parameters, making them more efficient overall.

Although the conditional computing aspect of MoEs presents a clear pathway to mitigate the computational scaling that accompanies the enlargement of models, it is essential to acknowledge that deploying MoEs on conventional artificial intelligence (AI) hardware encounters similar challenges as found with their dense counterparts. The scaling of MoEs, whether by increasing the number of experts or the embedding size, will inevitably lead to increased data movement and thus to additional energy and time penalties. This issue stems from the hardware adhering to the von-Neumann architecture. GPUs, for instance, require continuous shuffling between the various levels of memory hierarchy and the computational units (this is also referred to as the von-Neumann bottleneck). This imposes a ceiling on the achievable computational efficiency and a substantial floor to the required power budget¹⁷.

A promising alternative to the von-Neumann architecture that has risen in popularity is in-memory computing using non-volatile memory (NVM) devices^{18–21}—commonly referred to as analog in-memory computing (AIMC). AIMC accelerators leverage device characteristics and circuit laws to perform matrix–vector multiplication (MVM) operations within the memory arrays storing the weights, thus eliminating the necessity for weight shuffling, and requiring only the movement and buffering of activations. Moreover, the operation is performed in the analog domain, resulting in energy-efficient computation with $\mathcal{O}(1)$ time complexity. Recent publications of large-scale integrated AIMC

chips, using phase change memory (PCM)^{22,23}, resistive RAM (ReRAM)^{24–26} and Flash²⁷, show the viability of the technology. However, for AIMC to be considered for LLM inference, such chips must be capable of storing the entirety of the model’s weights in the NVM devices. Although dense integration of NVM devices has been demonstrated in the literature²⁸, current AIMC chips are orders of magnitude away from the parameter count of LLMs. To that end, the advancement from two-dimensional (2D) planar NVM to three dimensions offers a native solution to the aforementioned problem. By scaling to the third dimension, the demonstrated density of such memories^{29,30} provides a viable path towards storing the parameters of an LLM model. The potential of this technology has attracted the attention of researchers, leading to proposals for 3D AIMC architectures for deep-learning inference and vector similarity search^{31–35}.

In the following, we combine the conditional computing mechanism of MoEs with the characteristics of 3D AIMC. We demonstrate how the concept of conditional computing of MoEs is an ideal fit for 3D AIMC and how MoEs scale favorably compared to dense networks. We simulate the execution of dense and MoE-based LLMs in an abstract 3D AIMC system and compare their scaling behavior. It should be noted that providing an exact accelerator architecture is beyond the scope of this work. Instead, we investigate the impact of the inherent limitations of 3D AIMC hardware on different network architectures, gaining insights into the design principles of such an accelerator. Furthermore, we study the impact of noise introduced by the analog hardware on a language modeling task and demonstrate floating-point arithmetic-equivalent accuracy (iso-performance) at noise levels comparable to those observed in previously published AIMC hardware.

Results

The simulation framework

There are several reasons motivating the implementation of MoEs on 3D AIMC hardware. In Fig. 1b we present the number of non-stationary (for example, activations) and stationary (for example, weights) parameters of several MoEs for different sequence lengths. The memory associated with storing weights outweighs the memory required for buffering activations by up to three orders of magnitude. Moreover, the demonstrated densities of 3D NVM have reached more than 10 Gb mm^{−2} (Fig. 1c). If 3D AIMC hardware would approach the 3D NVM densities, all the parameters of a 100-billion-sized LLM model could be stored in a single AIMC chip.

To perform an MVM $\mathbf{y} = \mathbf{x}\mathbf{W}^T$, one programs the elements of the weight matrix, $\mathbf{W}$, onto the devices that correspond to a 2D slice of the memory stack, herein referred to as a tier. During inference, the target tier is selected and the input vector $\mathbf{x}$ is fed into the memory array. The output vector read back from the tile will be the result of the computation $\mathbf{y}$. The computation is typically performed in the analog domain by exploiting the physical attributes of the memory devices and their array-level organization (Fig. 1d). When deploying MoEs, the weight matrices in each expert are chunked and mapped to tiers in separate tiles. This way, the weights of the FFN in each expert are mapped to the same set of tiles, and the experts are stacked vertically (Fig. 1e). Note that, although the use of 3D memory alleviates capacity constraints, it does not necessarily increase the computational density of the accelerator. This is because performing MVM in parallel across multiple tiers on one tile is impractical. In some cases, this limitation—here referred to as the one-tier-at-a-time (OTT) constraint—is introduced by the physical arrangement of the memory devices³¹, whereas in other cases it implicitly arises as the amount of scratchpad memory and peripheral circuits required for supporting parallel MVM and would prohibitively increase the areal footprint. Therefore, these tiles need to be operated in a mutually exclusive way, which can have a detrimental impact on inference performance if not carefully considered.

We base our comparison of dense and MoE-based models on the Mixtral 8×7B¹ architecture, which is a decoder-only variant of the

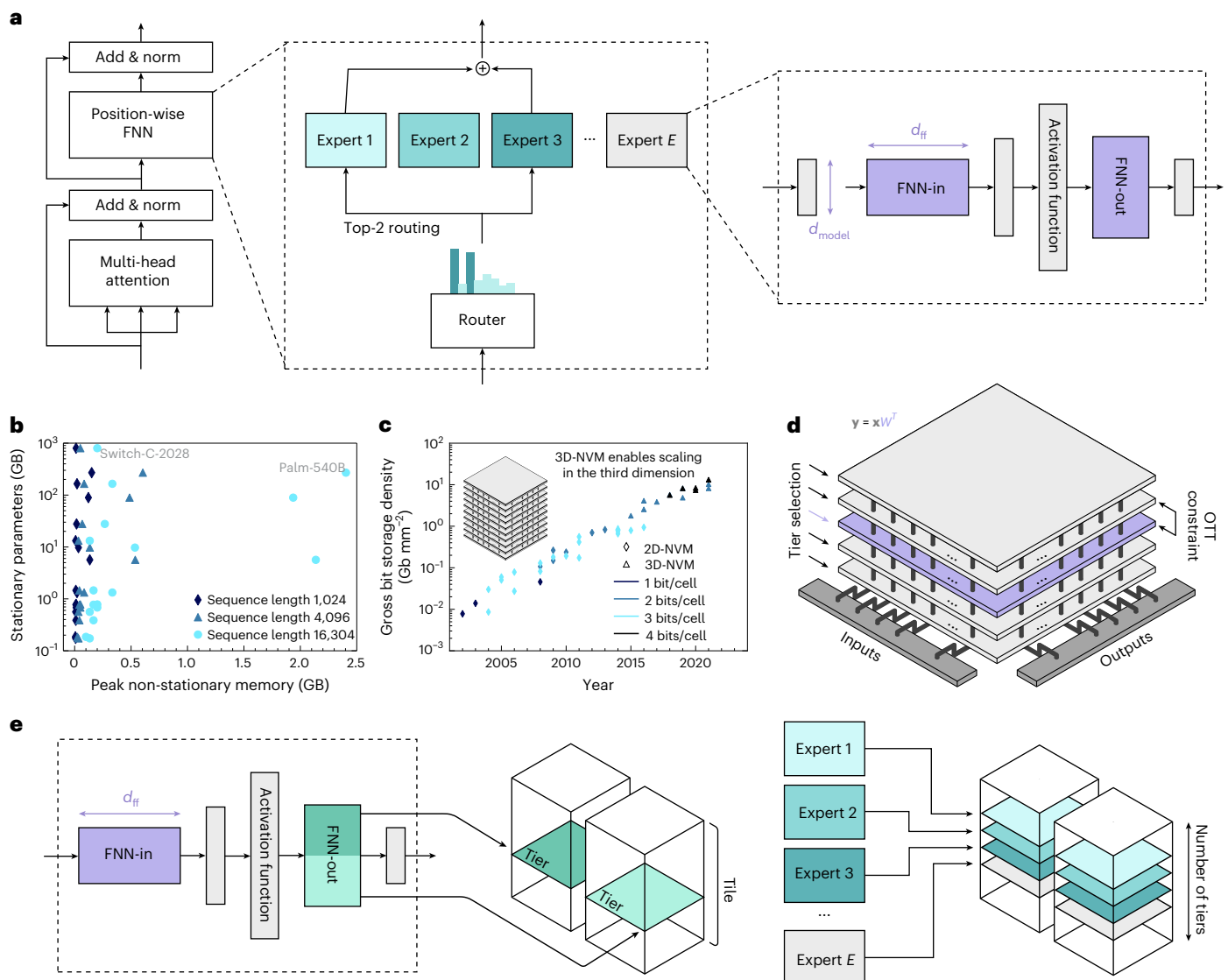


Fig. 1 | Three-dimensional AIMC-based implementation of MoEs. **a**, MoEs typically replace the FNN blocks in conventional transformers with multiple expert FNNs. In the forward pass, the router first determines for each token the experts that will be used for processing this token. Here, top-2 routing is illustrated, meaning that the router identifies two experts that process the token. **b**, As the model sizes of LLMs steadily increase, the memory requirements for stationary parameters (weights) dominate the non-stationary memory requirements (activations). **c**, Illustration of the increase in storage density over the years as a result of stacking dense NVM arrays vertically^{60,61}. **d**, Illustration of an abstract 3D AIMC tile. The dark rectangles depict input and output circuitry.

The stacked square boxes indicate the vertically stacked tiers, each comprising an analog crossbar. The purple box indicates the activated tier. The black strings illustrate vertical connections. When computing the MVM $y = xW^T$, the tier holding the weights W is selected and input vectors x are fed into the array. The resulting output vectors y are read out. Performing MVM in parallel across multiple tiers is often not feasible, and we refer to this as the OTT constraint. **e**, The experts, which are typically a two-layer FNN, are stacked on top of each other. In the case where the weight matrix of one of the layers of the FNN exceeds the tier dimension, it must be split into chunks, which are distributed across multiple tiles. Note that these chunks can also be stacked on top of each other.

original switch transformer¹¹. The MoEs in our comparison substitute every feedforward block with a MoE layer, comprising E experts and a router. For the comparison, we vary the number of parameters of the dense and MoE-based models by increasing the model dimension, referred to as d_{model} . We also increase the number of experts, which allows the MoEs to surpass the dense models in the number of parameters, reaching up to ~51 billion parameters. An extensive list of the hyperparameters used in the comparison is provided in Extended Data Table 1.

To assess the performance of each model, we developed a high-level performance simulator, an overview of which is presented in Fig. 2. With this simulator we can deploy models on abstract 3D AIMC accelerators (Fig. 2a) comprising a number of AIMC tiles, five digital processing units

(DPUs) and four multi-head attention (MHA) units. These are utilized to perform the MVM, light digital processing and multi-head attention computation, respectively. For each model, we constructed an accelerator with the number of tiles necessary to fit the weights of the model, keeping the number of vertical memory tiers constant. As shown in Extended Data Table 2, for all digital operations we chose the latency and energy numbers as reported in ref. 36. However, the MVM latency and energy values were chosen to be slightly higher than that reported in ref. 36 to account for the potentially higher numbers associated with 3D AIMC compared to 2D AIMC. More details on the accelerator are provided in Methods section ‘The 3D AIMC accelerator’.

Following the definition of the accelerator, the simulation framework proceeds to the mapping stage. The linear layers of the model

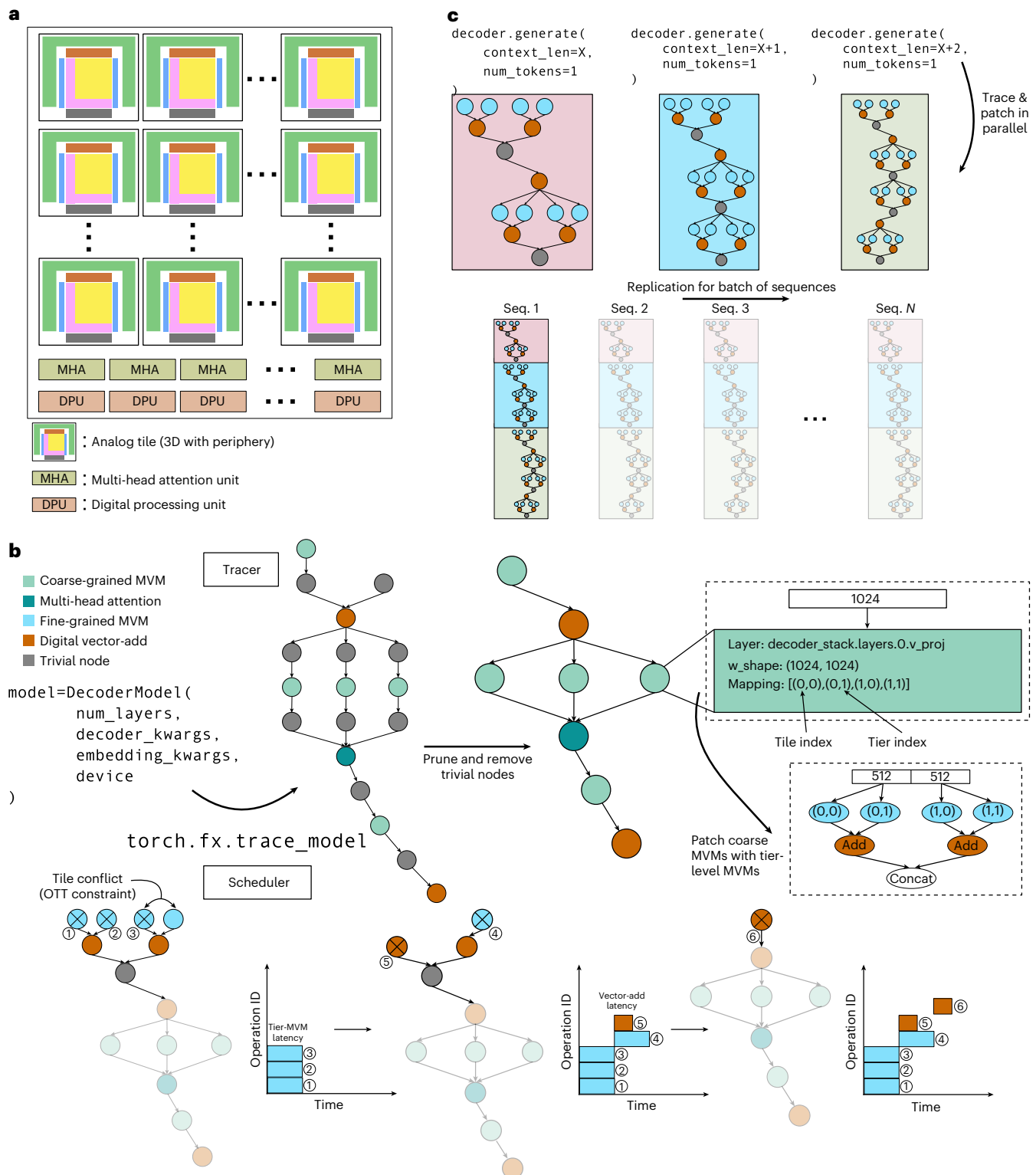


Fig. 2 | Overview of the simulation framework. a, The simplistic hypothetical 3D AIMC accelerator is composed of 3D AIMC tiles, MHA units and DPUs. **b**, After defining the model using custom modules built on PyTorch, we map the network to the accelerator. We then use torch.fx to extract the computational graph, where nodes represent operations, and incoming edges into each node represent the data dependencies. After pruning irrelevant nodes from the graph, MVM nodes are replaced with sub-graphs representing the dataflow of a chunked MVM, where partial MVM is executed on the different tiers and tiles. After the graph has been prepared, the scheduling commences, where we iteratively search the graph for source nodes. If no constraints prevent the source nodes from being executed, they are scheduled and executed. This process is

repeated until the graph is empty. **c**, We generate the graph for generating one token given a context length that varies from 1 to the maximum context length. By concatenating these sub-graphs vertically, the final graph representing the autoregressive decoding loop is obtained. To account for batch sizes greater than one, we replicate the graph horizontally. It should be noted that while we assume a batch size greater than 1, we do not model premature generation of the end-of-sequence token of individual sequences, but rather generate a fixed number of new tokens for every input sequence in the batch. The strategy for scheduling source nodes is depth-first, meaning that nodes belonging to sequence X are favored over nodes belonging to sequence $X + 1$.

are mapped to the tiles of the accelerator using a greedy in-order layer-utilization-based methodology (Methods section ‘Simulation of LLM execution on the 3D AIMC accelerator’). After the mapping, the simulator generates the token-wise operation graph using torch.fx (Fig. 2b). In decoder-only models, the input sequence is directly fed into the model, and a new token is generated and appended to the sequence. This extended sequence is then again fed into the model until an end-of-sequence token is generated.

We focus on batched autoregressive generation of tokens given an input sequence. We focus on this use-case because it is currently the most popular way to interact with an LLM. However, other use-cases exist, such as prompt or multimodal processing³⁷. These set-ups typically utilize slightly different architectures^{38–40}, which often include an encoder and a decoder. Although we focus on decoder-only architectures in this Article, we do not expect our main results to change for encoder-only or encoder–decoder architectures, as the underlying layer structure is—with the exception of the attention mechanism—the same.

Autoregressive generation of tokens can be greatly accelerated by using KV-Caching⁴¹. This is done by only requiring the last token to be fed into the decoder, rather than the entire sequence each time. This introduces a compute–memory trade-off, the impact of which is substantial in performance and memory requirements, especially considering that the size of the KV-Cache scales linearly with batch size, sequence length, model dimension and number of layers⁴¹. Using KV-Caching necessitates the addition of a non-negligible amount of dynamic random access memory (DRAM). Because of the high capacity and weight stationarity of 3D AIMC, DRAM technologies, which have less capacity, are cheaper and are more efficient, can be considered. When DRAM is added to a 3D AIMC system, various trade-offs between re-computation and caching, and the subsequent effect on latency, throughput, external memory requirements and efficiency can be studied. It is worth noting that in its basic application, KV-Caching can be employed in AIMC systems in the same manner as in digital accelerators. Although the main results presented in this Article have been obtained without KV-Caching, we have quantified the effect of KV-Caching on throughput and energy efficiency in separate experiments.

When batched inference is considered, we construct the execution graph for a single sequence and replicate it for the required number of sequences in the batch (Fig. 2c). Furthermore, the simulator natively supports inter-sequence pipelining, meaning that operations associated with one sequence can be interleaved with operations of another. We observe that although a batch size greater than 1 generally raises the peak scratchpad memory requirements, inter-sequence pipelining notably reduces the total inference time (Supplementary Note 1 provides a more thorough study). Finally, after the execution graph is generated, the simulator schedules the individual operations, providing the total execution time required for a workload, along with the energy consumption and peak scratchpad memory requirements (Methods section ‘Simulation of LLM execution on the 3D AIMC accelerator’).

Comparing the scaling behavior of dense and MoE-based models

In the experiment depicted in Fig. 3a, we study the scaling behavior of dense and MoE-based models deployed on the 3D AIMC accelerator by varying the model dimension d_{model} and the number of experts. For each d_{model} , the number of tiles is chosen as the required number of tiles for hosting the MoE-based model with the most number of experts and the respective d_{model} . The alternative to this choice is that the number of tiles is chosen as the minimum number of required tiles per model, which makes the results harder to interpret, as the effective number of OTT conflicts depends on the number of tiles available for mapping the model (Supplementary Note 2 provides a more detailed investigation). By increasing the model dimension d_{model} , the inference time

increases substantially, as a larger d_{model} directly increases the number of FLOPs and contributes to more OTT conflicts in the pipeline. This is because the layers increasingly conflict with other layers mapped to the same tiles. The inference time increases sharply when moving from $d_{\text{model}} = 512$ to 768. This jump in inference time can be attributed to a sudden increase in the total number of OTT conflicts. We generally observe that our inference time is dominated by the overall sum of OTT conflicts across all tiles. Although the number of OTT conflicts per tile even decreases when moving from $d_{\text{model}} = 512$ to 768, the total number of tiles necessary to fit the largest MoE model increases drastically, as one 768×768 layer now occupies four tiers, compared to one tier for the 512×512 -layer size. From this, we also conclude that the MVM operations performed in the analog tiles contribute a non-negligible fraction of the overall latency. Supplementary Note 3 further investigates how much each operation contributes to the overall latency and energy, and how these proportions change on varying parameters such as sequence length and model size.

Unfortunately, scaling dense models to the billion-parameter regime inevitably means increasing d_{model} to exorbitantly high values⁷. MoEs, on the other hand, can be scaled by increasing the number of experts, keeping the FLOPs constant. Figure 3a shows that we can easily scale to 51 billion parameters at the same inference time as the dense model with the same model dimension. By scaling the model using more experts, we make ideal use of the high capacity of 3D NVM, while avoiding additional OTT conflicts.

On the other hand, von-Neumann architectures, such as GPUs, do not exhibit the constant scaling of 3D AIMC. With an increase in model dimension d_{model} and the number of experts, the volume of data associated with the weights of the experts that need to be fetched from memory—often DRAM—increases. Figure 3b shows the amount of expert weight data that needs to be fetched to generate eight new tokens from a context of length 64 as a function of the number of experts for four distinct model dimensions d_{model} , assuming the use of 5-bit weights. Methods section ‘Calculation of weight movement for MoEs’ provides more details about how the amount of moved memory is computed.

In 3D AIMC there is no need to fetch weights from memory, so the overall peak memory consumption barely exceeds 1 MB (Fig. 3c). The peak memory originates from the staged data at the MHA layer, which is roughly four times the sequence length times the model dimension (Q, K, V and 1 for the residual connection). Here we consider a batch size of 4, and hence the aforementioned activations must be staged at most four times in parallel. For a model dimension of 1,024 and a maximum sequence length of 72, this results in ~1.18 MB. We assume int8 activations and do not use KV-Caching. Although the inference time comparison was done for top-1 routing, meaning that only one expert is picked for each token, recent studies^{13,16} have shown that increasing the number of utilized experts k improves model accuracy. This comes at the cost of an increased number of FLOPs, which is directly reflected in the inference time, as Fig. 3d illustrates. Noticeably, as the FLOPs in the expert FFN start to dominate the total number of FLOPs, the factor by which the inference time increases approaches k . From this, we conclude that it is generally better to reduce the model dimension of the experts while choosing a higher k , which has been found to increase model accuracy¹⁶.

Comparison against GPUs

We compare the hypothetical 3D AIMC accelerator with the NVIDIA A100 GPU⁴², in terms of throughput, area efficiency and energy efficiency. We perform comparisons across various generative dense models, including GPT-2⁴³ in different scales (300 million, 700 million and 1 billion) and Megatron 1 billion. We also extend our comparison to the small and base variants of a decoder-only variant of the switch transformer (denoted ‘MoE’). The models are selected so that they fit within the DRAM of one A100 GPU, allowing us to neglect the overhead

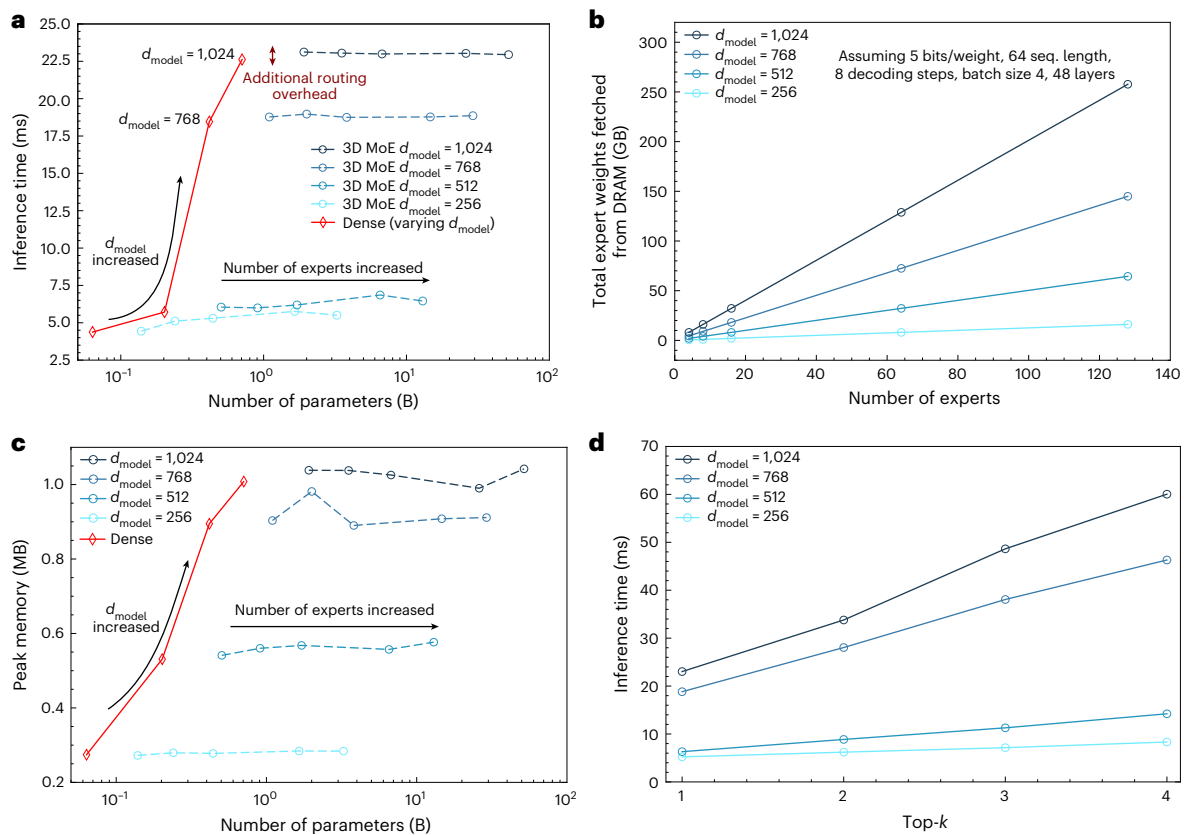


Fig. 3 | Inference performance of MoEs deployed on 3D AIMC. a, Inference time as a function of model size for both conventional and MoE models. Conventional transformer models scale by increasing the size of the weight matrices, and it can be seen that this increases the per-MVM latency due to the OTT constraint. On the other hand, MoEs scale by increasing the number of experts, which can be simply stacked vertically. **b**, When implemented on a conventional digital accelerator such as a GPU, the total expert weights that need to be fetched from DRAM as a

function of the number of experts and d_{model} . **c**, The peak memory (in MB) without KV-Caching for a maximum sequence length of 72, shown as a function of the number of model parameters. Even though the memory requirement increases with d_{model} , the worst-case memory requirement is still ~ 1 MB. **d**, Inference time as a function of k for the selection of top- k routing. It can be seen that increasing k results in an increase in the number of FLOPs for each MoE layer, which in turn increases the inference time.

of model parallelism over multiple GPUs. All models are assumed to have a vocabulary size of 50,000 and are executed on four sequences with a sequence length of 64, generating eight new tokens. Details on the model architectures and number of tiles/tiers used are provided in Extended Data Table 4. To provide a fair comparison, we calculate upper-bound estimates of the achievable throughput in the A100, based on whether a model is compute- or memory-bound. When a model is calculated to be compute-bound—the time spent on computation is larger than the time spent on memory transfers—we calculate the throughput based on the total number of FLOPs in the model. More specifically, we assume the int8 data format, resulting in a presumed throughput of 624 tera operations (TOPs) s^{-1} at 400 W. As 100% model FLOPs utilization (MFU) is practically impossible to achieve, we additionally report the numbers at a more realistic 70% MFU. On the other hand, when a model is calculated to be memory-bound—the time spent on transferring data is larger than the time spent on computation—we calculate the throughput based on the total number bytes that need to be transferred at peak memory bandwidth (Methods section ‘Upper-bound estimate of throughput for GPU’).

Our findings reveal that, for smaller model sizes, the throughput of the A100 is generally one order of magnitude higher compared to the 3D AIMC accelerator (Fig. 4a). For larger MoEs, the total amount of weights that need to be fetched from DRAM increases drastically, and the total inference time is now lower-bounded by the time it takes to fetch these weights at a bandwidth of $1,555\text{-GB s}^{-1}$. As a result, the throughput of the 3D AIMC accelerator is up to six times higher than

that of the A100. This result directly translates into the area efficiency, and we show that 3D AIMC is consistently more area-efficient (up to $20\times$) than the A100 (Fig. 4b), for which we assume a total area of 826 mm^2 (ref. 42). For the 3D AIMC accelerator, we assume a footprint of 1 mm^2 per tile, 5 mm^2 per MHA unit and 1.25 mm^2 per DPU. For large models, we observe that the tile size dominates the overall area, which is still very reasonable owing to the 3D nature of the tiles. Finally, the 3D AIMC accelerator outperforms the GPU by up to three orders of magnitude in energy efficiency (Fig. 4c). This assumes a power consumption of 400 W at 100% MFU and 301.93 W at 70% MFU (calculated using a measured static power consumption of 73.1 W) for the GPU.

GPUs can truly leverage their high throughput when the batch size is large. Because the 3D AIMC architecture we consider in this work does not perform operations in a single instruction multiple data (SIMD) fashion, it is expected that the GPU will perform increasingly better when the batch size is increased. We present a comparison between 3D AIMC and a GPU for larger batch sizes in Supplementary Note 4. We show that for the ‘MoE (dec-only, base)’ architecture, the GPU has higher throughput for batch sizes ≥ 32 . However, 3D AIMC still shows orders of magnitude higher energy efficiency than the GPU.

By using KV-Caching, the decoding speed can be greatly increased. In Supplementary Note 5 we report additional experiments that compare the performance of 3D AIMC with the NVIDIA A100 when KV-Caching is employed. We find that for the setting considered in Fig. 4, the throughput of 3D AIMC slightly deteriorates compared to the GPU. However, 3D AIMC is still orders of magnitude more

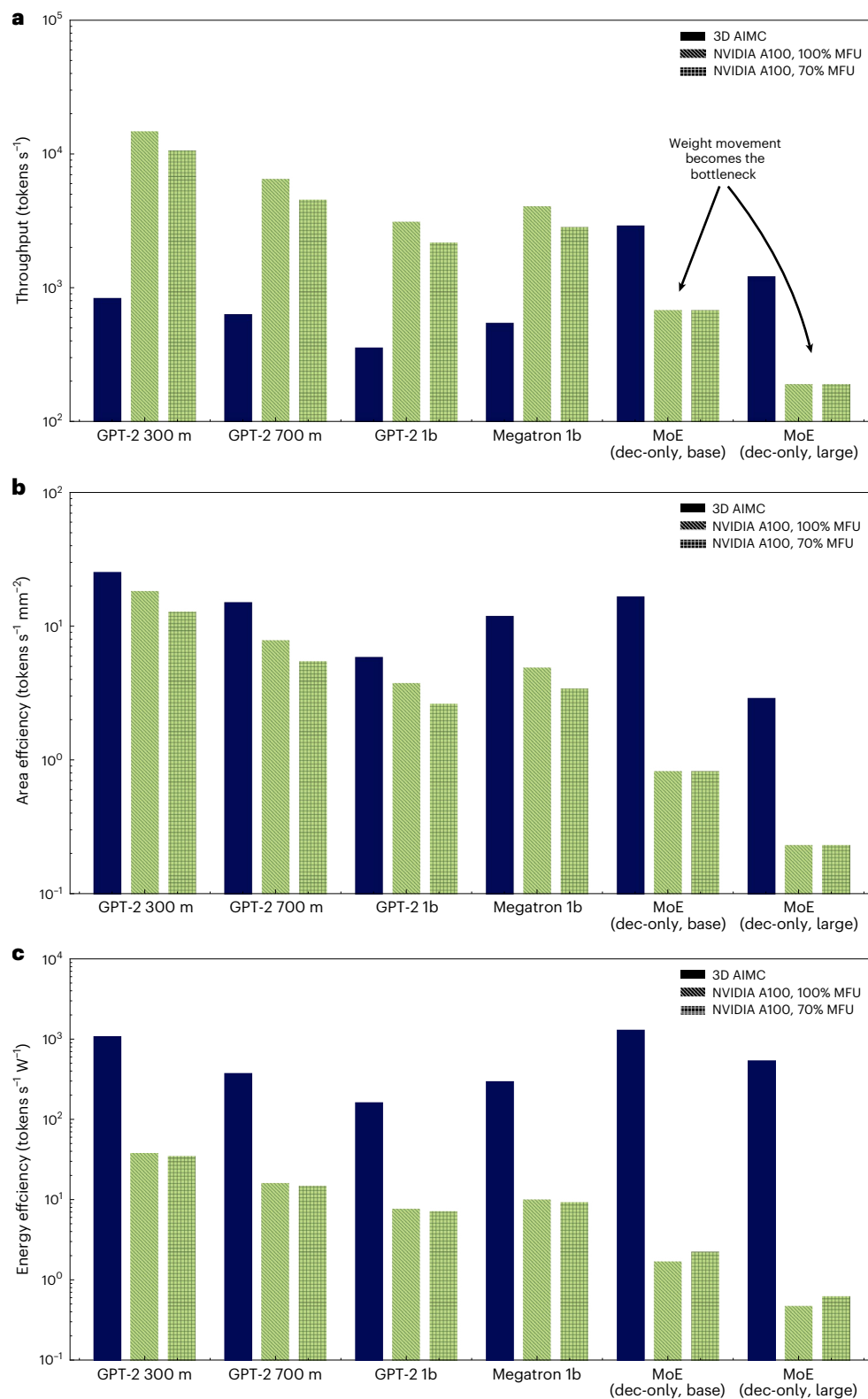


Fig. 4 | Comparison of throughput, area efficiency and energy efficiency between simulated pipelined execution of LLMs on 3D AIMC against the NVIDIA A100. Different model architectures are compared: GPT-2⁴³ with 300 million, 700 million and 1 billion parameters, Megatron⁶² with 1 billion parameters and two MoEs (see Extended Data Table 4 for architecture details). **a**, Throughput (measured in generated tokens s^{-1}). The GPU is mostly compute-bound for smaller dense models (assuming 624 TOPs s^{-1} at 100% MFU), but it

is memory-bound for larger MoEs (assuming a bandwidth of 1,555 GB s^{-1}). For larger models, 3D AIMC has the advantage that weights do not need to be moved around, yielding an overall higher throughput. **b**, Area efficiency (measured in tokens $s^{-1} mm^{-2}$). The area for the GPU is 826 mm^2 and does not include the DRAM footprint used for storing the weights. **c**, Energy efficiency (measured in tokens $s^{-1} W^{-1}$). At 100% MFU, the peak power consumption of 400 W is assumed, while at 70% MFU, 301.93 W is assumed. dec, decoder.

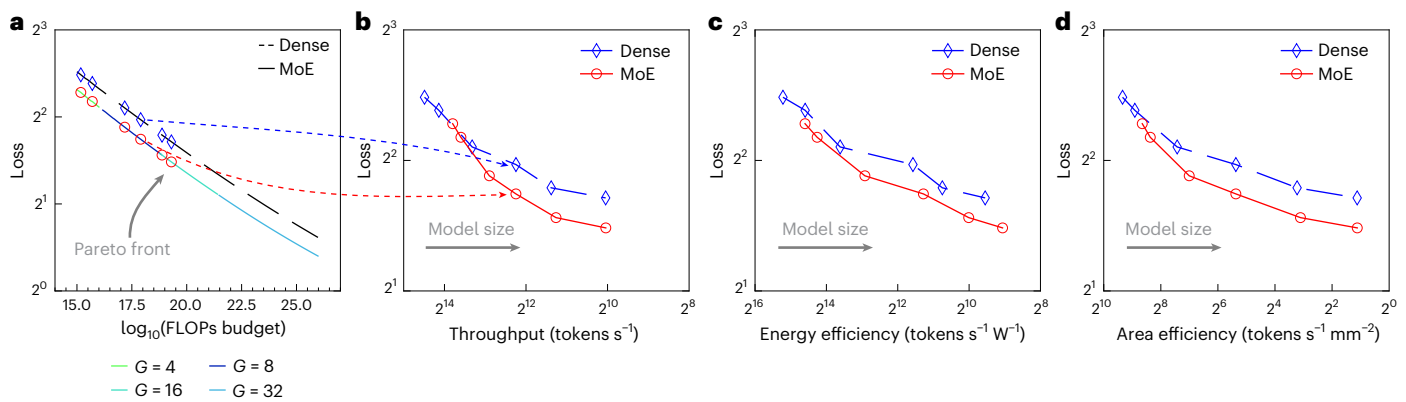


Fig. 5 | MoEs form the Pareto front in terms of model accuracy against system performance when compared to dense models. a, For a given FLOPs budget, MoEs always achieve a better model accuracy (loss) compared to dense models. As the number of training FLOPs increases, it is better to have many small experts as opposed to a few large experts. This is controlled by the granularity

G , which steadily increases. **b**, When a subset of the architectures from **a** are evaluated on our accelerator, one can see that MoEs form the Pareto front in terms of throughput. **c, d**, The same also holds for energy efficiency (**c**) and area efficiency (**d**).

energy-efficient than the GPU, even for scenarios where many more tokens are generated (up to 512).

MoEs form the system performance Pareto front on 3D AIMC

From the above experiments we can conclude that 3D AIMC achieves substantially better energy efficiency than the NVIDIA A100, and—depending on the experiment setting—even better throughput and area efficiency. This difference in system performance is especially strong for MoEs. It is, however, not clear whether MoEs should be favored over dense models when the relation between model accuracy and system performance is taken into account. To answer this question, we relate network accuracy to system performance for both MoE and dense models and compare their trends. To ensure a fair comparison, one needs optimally trained MoEs and dense models that were trained given the same training FLOPs budget and the same training data. Instead of training these models ourselves, we used scaling laws for MoEs¹⁵ and dense models⁶. Scaling laws are functions that relate a fixed FLOPs budget to the loss a specific network architecture can achieve, when trained under optimal conditions. Using these scaling laws, we can extract optimal MoE and dense model architectures that we can then deploy on our accelerator (Methods section ‘Using scaling laws for the accuracy-performance trade-off analysis’). Figure 5a shows that—as shown in ref. 15—MoEs always outperform dense models given a fixed budget of training FLOPs. When a subset of the architectures from Fig. 5a are evaluated on our accelerator, MoEs consistently show better system performance for a given model accuracy (Fig. 5b–d). We conclude that for any network accuracy, choosing an MoE over a dense model will always yield better system performance when deployed on 3D AIMC hardware.

Robustness of MoEs to AIMC noise

Recent studies¹⁶ and model releases¹ have shown that MoEs can be trained to perform on par with, or even outperform, dense models using a smaller number of active parameters. However, these results were obtained for computations using floating-point precision, and it is not clear whether they hold for computations done using noisy AIMC hardware. To study this, we trained a MoE¹⁶ and a dense model for language modeling on the WikiText-103 dataset⁴⁴. We then used hardware-aware training (Methods section ‘Hardware-aware training of MoE-based LLMs’) to obtain more robust versions of both networks. Using perplexity and the ROUGE-1 score, we compare the FP-32-based model against the hardware-aware trained models with different magnitudes of Gaussian noise injected into the weights. We use an abstract

noise model in these simulations, because the exact noise characteristics of the devices that will be used for 3D AIMC are currently unclear. In Fig. 6a we show that, for the MoE-based model, iso-performance is retained up to noise levels of 6.3%, which is within the range of noise values observed for previously published AIMC hardware²². For reference, the programming noise of a PCM based AIMC chip^{22,45} is—for this network—equivalent to 4.75% of additive Gaussian noise. Note that this comparison merely serves as a way to map the abstract Gaussian noise to a realistic noise magnitude. The noise levels of 3D AIMC hardware will be dependent on the maturity of the device technology, and future hardware demonstrations in the literature will provide a realistic magnitude for evaluating model accuracy in such systems. Figure 6b shows the perplexity on the test set of WikiText-103 at different magnitudes of Gaussian noise injected into the weights before inference. Here, the MoE-based model slightly outperforms the dense network, and the gap is mostly retained across all noise values we tried.

Discussion

The primary focus of this work is to introduce readers to the combination of 3D AIMC and the concept of conditional computing. With the findings of this work, we aim to stimulate architectural exploration towards the development of 3D AIMC accelerators capable of competing with conventional AI accelerators. We have identified several key areas for this exploration. First, it is crucial to identify a viable communication fabric that can efficiently move activation data without limiting the throughput of the compute elements. Various approaches have been proposed for 2D AIMC systems, and it is imperative for the community to assess their viability and scalability in the context of 3D AIMC. Next, an essential aspect of any generative transformer accelerator is the organization of its memory hierarchy. Although 3D AIMC has a clear advantage over von-Neumann systems due to weight stationarity, there is a study to be conducted on how to localize the scratchpad memory within the accelerator and whether or how to utilize high-bandwidth external memory for scaling to large sequences and for memory-compute trade-offs like KV-Caching. Furthermore, the precision, amount and complexity of the digital auxiliary compute is crucial to ensure minimal impact on the bandwidth and efficiency provided by the tiles. Finally, in this work we assume that embeddings are stored and retrieved from the tiles through one-hot MVMs. It remains to be determined whether the tiles can support such sparse reads or if embeddings should be fetched from an external memory.

Simulating systems at a fine-grained level can yield precise results, but it risks obscuring key findings due to design flaws, and can increase

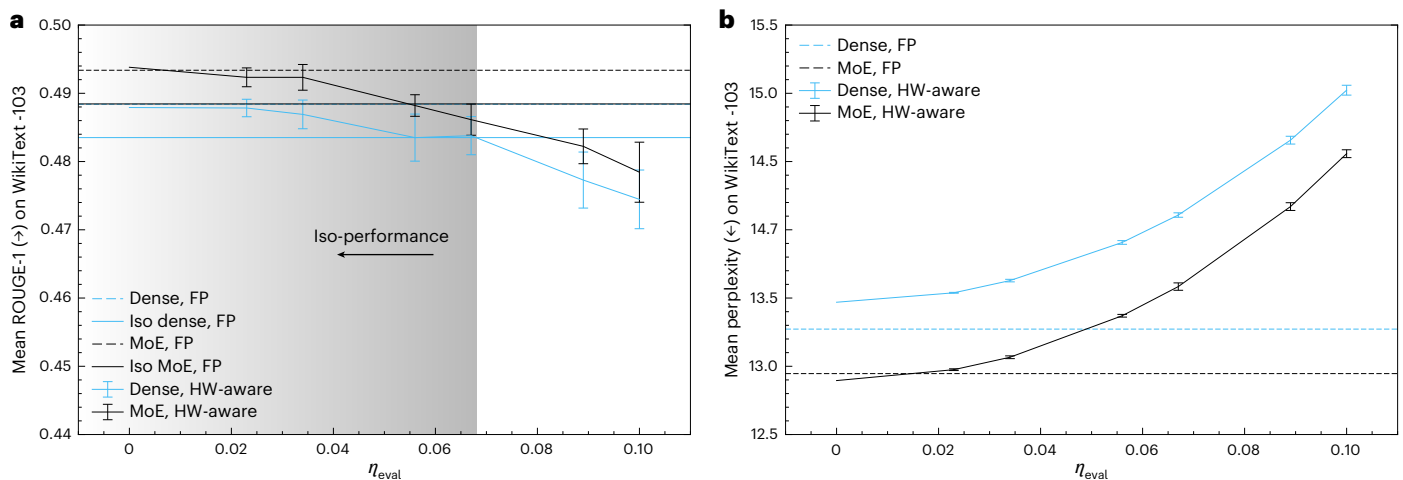


Fig. 6 | Impact of AIMC noise on iso-performance. a, Mean ROUGE-1 score plotted as a function of the additive Gaussian noise injected into the weights before inference. We use the MLPerf definition of iso-performance in terms of accuracy (99% of the FP-32 baseline) to determine the cutoff noise magnitude for the MoEs. It can be seen that with hardware-aware training, we can retain the iso-performance for noise magnitudes comparable to the AIMC noise values

reported in the literature²². Error bars represent the standard deviation around the mean over 11 inference runs for different noise samples added to the weights. HW, hardware. **b,** Mean perplexity plotted as a function of additive noise. It can be seen that the MoE model performs better than its dense counterpart (lower perplexity). Moreover, with hardware-aware training, we can maintain this performance gap across noise levels.

experiment time. In this study, the chosen level of abstraction was sufficient to show that MoEs are better suited than dense models for 3D AIMC hardware. However, to study the impact of choices such as communication scheme or memory layout, a more fine-grained simulator, capable of simulating a more detailed architecture, would be beneficial.

Regardless of the memory technology, a key challenge in deep-learning inference with AIMC is synaptic weight fluctuation due to device noise, which necessitates hardware-aware training. Other memory-dependent non-idealities, such as conductance drift and device-to-device variability, also come into play, but are similar to those encountered in 2D AIMC. Notable progress has already been made in addressing these issues through large-scale demonstrations of 2D AIMC architectures^{22,23}, and these solutions are expected to be applicable to 3D AIMC as well. However, there could be additional non-idealities specific to 3D AIMC, such as increased *IR* drop, larger *RC* time constants, and inter-tier interference, which will require further investigation as the technology matures.

Methods

The 3D AIMC accelerator

The accelerator consists of a parameterizable number of tiles, each composed of a configurable number of tiers. Two types of digital compute are included, with the first type handling lightweight digital computation, such as accumulations, activation and normalization functions, and the second type handling heavier digital computations associated with MHA (Extended Data Table 3 presents the operations we consider and where they are executed). Data transmission between tiles and digital compute units is facilitated by an interconnect with a constant time and energy cost per bit. These simulations do not consider interconnect contention, as it has a strong dependence on the selection of the interconnect technology and the physical layout of the accelerator. Data exchange between digital units occurs through memory, meaning that the transferred data have to be written into a buffer before another digital unit then reads and processes said data. All digital units are considered to have access to a scratchpad memory, from which they read inputs and write outputs. The read/write latency and energy associated with these static random access memory (SRAM) interactions are included in each digital operation. We refrain from localizing SRAM to specific units and assigning a capacity for each block, as more detailed studies are required to determine the optimal

memory layout for such an accelerator. Nevertheless, the simulator can record the number of vectors buffered in scratchpad memory for each time step, providing the peak memory required to execute a given workload. The intermediate activations are assumed to be in int8, so digital operations are also carried out in int8.

Simulation of LLM execution on the 3D AIMC accelerator

To assess the performance and energy efficiency of LLM inference in the 3D AIMC accelerator, we developed a high-level, architecture-agnostic inference simulator tailored to the pipelined execution flow of AIMC accelerators. The simulator has been functionally verified (that is, it produces the same logits as pretrained networks) and validated against a previously published low-level simulator of a 2D heterogeneous AIMC architecture (details are provided in Supplementary Note 6). Implemented in Python, the simulator leverages existing libraries such as PyTorch⁴⁶ and torch.fx⁴⁷, which ensures high flexibility and ease of extension. Under the hood, it operates on the simple torch.fx intermediate representation (IR) in lieu of more complex and less scalable IRs, such as torch.jit.script. The process of simulating a network in our simulator involves four key steps. First, we define the network architecture. Second, we map the layers of the network to 3D AIMC tiles. We then trace the network to generate the operation graph in the torch.fx IR. Finally, we simulate its execution by scheduling the operations of the graph. The outputs of the simulator include the inference time for a given number of sequences and sequence length, the energy expended in performing all operations, and the peak memory required to buffer activations during inference.

To define the networks, we developed modified versions of PyTorch's elementary transformer blocks, namely the encoder and the decoder. Unfortunately, using the standard blocks and model definitions was deemed suboptimal due to the way torch.fx traces such models. When tracing conventional definitions, each torch module appears once at a per-tensor basis, meaning once, independent of the selected number of sequences and sequence length. Although this poses no problem when simulating conventional accelerators, the weight stationarity of AIMC necessitates a dataflow where each token (vector) is independent and passes through each operation at a different time. For instance, when tracing an encoder block, the conventional model's graph would force all tokens to pass through a first linear layer before proceeding to the next. In AIMC systems, however,

the execution flow would entail the first token passing through the first linear layer and then beginning on the second linear layer as the second token undergoes processing by the first linear layer and so on. The pipelined nature of the AIMC dataflow required us to modify the definition of the transformer modules to ensure that tracing yields a token-wise operation graph. Once these modules were updated, the definition of large models followed the standard approach of defining a PyTorch model with custom modules. To ensure correctness, we functionally verified the new modules against standard implementations of encoder–decoder and decoder-only transformers⁴⁸. Once a model is defined, the next step involves mapping the linear layers of the network onto the tiles of the accelerator based on a greedy in-order approach: for mapping a single weight matrix, we chunk the matrix into tile-shaped pieces (here 512×512) and greedily pick the set of tiles on the accelerator that have the lowest tile utilization. In this case, the utilization of a tile is defined as the sum of the utilization of each chunk belonging to some layer that has already been mapped to one of the tiers of the tile. It should be noted that the tile utilization does not simply correspond to the number of used tiers per tile, because the utilization of chunks belonging to expert layers is $1/E$, where E is the number of experts (assuming top-1 routing and uniform routing probabilities), while the utilization of non-expert layers is 1. Interestingly, this mapping strategy naturally leads to scenarios where expert layers are mapped on top of each other, as the summed utilization of all the experts equals the utilization of just one dense layer. We find that the utilization-based approach generally minimizes the number of conflicts arising from the OTT constraint (referred to as OTT conflicts). The network is traversed in the order of execution, and the layers are mapped in the order in which they are called. As we show in Supplementary Note 2, this mapping strategy leads to an almost ideal distribution of the computational workload across all tiles. It should be noted that—in favor of keeping our mapping and scheduling algorithms simple—tiles can only be occupied by one layer or parts of one layer. We note that co-occupation of tiles with different layers makes sense when the model dimension is smaller than the tile size, or when the model dimension is not dividable by the tile size.

For the embedding, we assume a vocabulary size of 50,000, where the embedding of one token is retrieved by performing an MVM with a one-hot encoded vector. Note, however, that through simple modulo arithmetic, one can identify the tiers required for retrieving the correct embeddings. This avoids performing an MVM with a 50,000-element vector. More details are provided in Supplementary Note 7. The third step involves tracing the model to acquire a token-wise operation graph in the torch.fx IR. The upper part of Fig. 2b illustrates this step. Initially, we utilize torch.fx to generate the initial operation graph. The resulting graph then undergoes processing and formatting to make it suitable for scheduling. This process entails removing trivial nodes that are not explicitly simulated (for instance, concat and view operations), while also incorporating nodes specific to our simulation (for example, communication nodes). Functional dependencies are preserved throughout this process, by ensuring causality is respected when connecting floating edges following the removal of a node. Finally, each node representing a linear layer that needs to be chunked and mapped across multiple tiers is replaced by a subgraph containing the tier-level mapped MVM operations and the requisite operations for combining the partial results. Finally, the operations in the processed graph are scheduled in a pipelined execution flow. To that end, we adopt a conventional instruction scheduling approach^{49,50}, during which we iteratively identify the source nodes in the operation graph, assess which nodes can be executed, based on the resource constraints of the accelerator, and designate them as processing. A global clock is then advanced by the time step required for any operation to be completed. Nodes that finish executing within this time step are removed from the graph, and the remaining time of all processing nodes is adjusted to account for the elapsed time. This process continues until all nodes are scheduled,

signifying the end of the inference. During this iterative process, we additionally calculate the energy required for each operation and determine the number of vectors that need to be stored in activation memory at each time step. An illustrative example of this process is depicted in the lower part of Fig. 2b.

The scheduling algorithm is configurable in two levels: (1) in terms of what it considers as resource constraints and (2) in the manner it selects an operation if multiple operations request the same resource simultaneously. Regarding the former, we take into account the OTT constraint, which occurs when multiple tokens request tiers belonging to the same tile, as well as constraints on the number of available DPUs and MHA units in the accelerator. It is worth noting that we elected to not enforce resource constraints on data transmission, as doing so would tether the simulations to a specific underlying communication channel and accelerator layout. The same principle applies to the activation buffer memory, where we choose to record the amount needed without applying a constraint on its placement or size. Regarding operation selection, we employ a heuristic that prioritizes operations from earlier sequences (lower sequence ID) and earlier tokens within each sequence (lower token ID). Through empirical observation, we find that this heuristic provides a solid baseline for achieving fast and resource-efficient inference.

In an initial version of the simulator, we traced over the autoregressive for-loop of generative transformers, which scaled poorly. We therefore resorted to another approach where we, in parallel, traced and pruned the graph for the generation of one token, given a context length of X , $X + 1$ and so on. Finally, we simply connected the graphs' endpoints to preserve the functional dependence of the autoregressive loop. Similarly, we replicated the graph horizontally to account for the execution of multiple samples in a batch. This process, which happens after the tracing, is depicted in Fig. 2c.

Calculation of weight movement for MoEs

To estimate whether the GPU is memory- or compute-bound, we calculate the total amount of memory associated with the moved weights of an MoE for a given use-case. To ensure that each expert's weights are moved just once, we assume to have sorted the tokens in the batched sequence according to the expert to which each token is routed. However, note that fetching of the experts must be repeated for every layer and iteration of the autoregressive loop. For a model dimension of 1,024, 128 experts, 48 layers and eight iterations of the autoregressive loop, this amounts to a total of ~250 GB of transferred data, taking ~0.16 s on an NVIDIA A100⁴². Here we assume a batch size of 4, but note that this does not have an influence on how often an expert is loaded, given that every expert receives at least one token in the sequence, which has a high probability given a long enough sequence and uniform routing probabilities. Equation (1) shows how we compute the amount of expert memory that is fetched from DRAM. The factor 0.625 corresponds to the number of bytes we need to store one 5-bit weight:

$$\begin{aligned} &\text{Total expert memory fetched (bytes)} \\ &= 0.625 \text{ number of generated tokens } n_{\text{layers}} n_{\text{experts}} 8d_{\text{model}}^2 \end{aligned} \quad (1)$$

This data movement incurs high energy consumption and an increase in inference time, although some of this could be hidden by performing computation while pre-fetching other weights. Because computation is done in-memory, and because of the high capacity of 3D NVM, this overhead is not present in 3D AIMC. In the context of KV-Caching, equation (1) would change as the number of experts n_{experts} that needs to be loaded changes to k , assuming top- k experts are selected per token. However, feeding only one token through the network—which would require only fetching one expert—would result in severe under-utilization of the GPU. To combat this, decoding steps making use of KV-Caching are combined across a batch of requests. This again leads to a scenario where many tokens are routed to different experts and—for large batch sizes—all experts need to be fetched again.

Upper-bound estimate of throughput for GPU

We estimate the maximum achievable throughput on the GPU based on whether the model is limited by computation or memory bandwidth. If a model is compute-bound (where computation time exceeds data transfer time), we calculate throughput using the total FLOPs, assuming the int8 data format, which yields an estimated throughput of 624 TOPs s^{-1} at 400 W. For memory-bound models (where data transfer time exceeds computation time), we determine throughput based on the total data that must be transferred at peak memory bandwidth. Based on this reasoning, the throughput can be calculated according to the following equation:

$$\text{Throughput}(\text{tokens } s^{-1}) = \frac{\text{Number of generated tokens}}{\max\left(\frac{\text{Total TOPs}}{\text{MFU } 624 \text{ TOPs } s^{-1}}, \frac{\text{Total weights moved}}{1,555 \text{ GBs } s^{-1}}\right)} \quad (2)$$

where total weights moved (in GB) is the amount of weights fetched from DRAM over the course of inference. This is calculated as the sum of the total expert memory (equation (1)) and the sum of the memory associated with the rest of the network weights. The total TOPs are calculated as the total number of add and multiply operations that occur during inference (Extended Data Table 3). Note that for models where the weights need to be stored in DRAM, the time spent on fetching the weights from memory is a generous lower bound on the inference time.

Using scaling laws for the accuracy–performance trade-off analysis

We base our experiments on scaling laws for MoEs¹⁵ and dense models⁶, as these scaling laws provide the optimal model architecture along with the best achievable loss for a fixed budget of FLOPs available for training. Using these scaling laws, we can compare optimal MoE and dense model architectures in a controlled setting, not disrupted by suboptimal choices such as using the wrong hyperparameters or too few tokens for training. We used the Chinchilla scaling law⁶, and the MoE-based scaling law presented in ref. 15. Here, we re-state both scaling laws. For the dense models, we have

$$\mathcal{L}(N, D) = c + \frac{a}{N^\alpha} + \frac{b}{D^\beta} \quad (3)$$

where N is the model size, D is the number of tokens in the dataset, and a, b, c, α and β are parameters that are fitted. For the MoEs, the scaling law can be formulated as

$$\mathcal{L}(N, D, G) = c + \left(\frac{g}{G^\gamma} + a\right) \frac{1}{N^\alpha} + \frac{b}{D^\beta} \quad (4)$$

where g and γ are additional parameters that need to be fitted, and G is the granularity. The granularity determines how many experts are chosen (like the top- k parameter), and is a free parameter ($G \in \{1, 2, 4, 8, 16, 32, 64, \dots\}$). Ludziewski and colleagues¹⁵ also introduced the expansion factor $E = 64$, which is the factor that determines by how much the number of parameters in a feedforward layer is scaled when we use an MoE layer instead of the classical feedforward layer. The number of experts is given by $N_{\text{experts}} = GE$. Inferring the optimal loss for a given budget of FLOPs F is the same as solving the following optimization problem:

$$\begin{aligned} \min \mathcal{L}(N, D, G) \\ \text{subject to: } \text{FLOPs}(N, D, G) = F \end{aligned} \quad (5)$$

Under the assumption that $d_{\text{model}} = 64n_{\text{layers}}$, one can reformulate this problem to a problem in a single variable, n_{layers} . From n_{layers} , the model FLOPs can be computed, and the number of training tokens can be solved for. Once reformulated, one can use the Brent method

to find the architecture that minimizes the loss. For the dense and MoE scaling laws, we use the parameters shown in Extended Data Table 5. For this experiment, we allocate the same number of tiles to the MoE and the dense model, although the dense model will require far fewer tiles. For the same experiment carried out under the condition where each model is deployed on the minimum number of required tiles, see Supplementary Note 8. For FLOPs budgets from 10^{15} to 10^{25} , we then solve for the optimal models in the MoE and dense space using the respective scaling laws. Using the optimal models, we then plot the FLOPs (x axis) against the losses (y axis). We confirm that the higher the FLOPs budget, the higher the optimal granularity G . This effectively means that, as the number of tokens and parameters increases, it is beneficial to have many small experts from which more are used during computation. So ‘top- k ’ is increased, while the expert dimension is reduced.

For a subset of FLOPs budgets, we then pick the respective optimal MoE and dense models and evaluate them on our accelerator. Given that the expansion factor in ref. 15 was 64, MoEs have generally more parameters, but most of these are inactive during computation. In these experiments we assume a batch size of 1 and one pass through the network, considering that larger set-ups would require a much longer simulation time, and we do not expect the comparative results to change.

Hardware-aware training of MoE-based LLMs

To make the models robust to noise, we follow the standard procedure⁵¹ of first training the FP-32 base model, followed by hardware-aware finetuning on the same dataset. In both phases, we train the networks for 100,000 steps on the document-level version of the WikiText-103 dataset, which was tokenized using a SentencePiece⁵² tokenizer with a vocabulary size of 8,000, which was generated using the training partition of the dataset. The base model used for training is an adapted version of the LLaMA model with rotary position embeddings that supports a variable amount of MoE layers. The MoE layer is programmed in triton⁵³ and adapted from ref. 16. The model supports all the base functionality from Hugging Face and is openly available (see Code availability section). The hyperparameters used for training the models are provided in Extended Data Table 6. For hardware-aware finetuning, we leveraged AIHWKIT⁵⁴, an open-source framework used for hardware-aware training of neural networks. Because AIHWKIT only supports standard PyTorch modules such as the convolutional or linear layer, we implement a custom analog MoE layer written in triton that supports most of the base hardware-aware training functionalities, including weight noise injection, output noise injection, input range learning and arbitrary input quantization. The correct forward and backward passes of the analog MoE layer are tested against a naive, linear layer-based implementation of the MoE layer, which can be easily handled by AIHWKIT. The hardware-aware triton-based MoE layer is also openly available (Code availability section) and is compatible with AIHWKIT⁵⁴ and AIHWKIT-Lightning⁵⁵. During training, we clip the weights to 2.0 standard deviations after every batch update and inject Gaussian noise $\zeta \sim \eta_{\text{train}} \max(|W|) \mathcal{N}(0, 1)$, where $\eta_{\text{train}} = 0.067$, into every weight matrix W in the forward pass. Although previous works⁵⁶ have demonstrated that sampling the Gaussian noise per sample in the batch improves downstream robustness, we sample the noise once per batch because it is much faster. We also use input range learning, where we initialize the input ranges using a moving average over the maximum of the inputs to every layer for the first 1,000 batches. We additionally quantize the inputs to 8 bits and use full precision in the backward pass. Embeddings and digital operations such as applying activation functions or performing MHA are computed in FP-32.

Reporting summary

Further information on research design is available in the Nature Portfolio Reporting Summary linked to this Article.

Data availability

Data were generated by the presented simulator and by evaluating the models trained on the WikiText-103 dataset. The WikiText-103 dataset is publicly available at <https://huggingface.co/datasets/Salesforce/wikitext>. Source data are provided with this paper⁵⁷.

Code availability

The code used to generate the results of this study is publicly available from <https://github.com/IBM/analog-moe> (ref. 58) and <https://github.com/IBM/3D-CiM-LLM-Inference-Simulator> (ref. 59).

References

- Jiang, A. Q. et al. Mixtral of experts. Preprint at <https://arxiv.org/abs/2401.04088> (2024).
- Touvron, H. et al. Llama 2: open foundation and fine-tuned chat models. Preprint at <https://arxiv.org/abs/2307.09288> (2024).
- Gemini Team Google et al. Gemini: a family of highly capable multimodal models. Preprint at <https://arxiv.org/abs/2312.11805> (2023).
- Brown, T. B. et al. Language models are few-shot learners. In *Proc. Advances in Neural Information Processing Systems* Vol. 33 (eds Larochelle, H. et al.) 1877–1901 (Curran Associates, 2020).
- Kaplan, J. et al. Scaling laws for neural language models. Preprint at <https://arxiv.org/abs/2001.08361> (2020).
- Hoffmann, J. et al. An empirical analysis of compute-optimal large language model training. In *Proc. Advances in Neural Information Processing Systems* (eds Koyejo, S. et al.) Vol. 35 (Curran Associates, 2022).
- Chowdhery, A. et al. PaLM: scaling language modeling with pathways. *J. Mach. Learn. Res.* **24**, 11324–11436 (2023).
- Jordan, M. & Jacobs, R. Hierarchical mixtures of experts and the EM algorithm. In *Proc. 1993 International Conference on Neural Networks (IJCNN-93-Nagoya, Japan)* Vol. 2, 1339–1344 (IEEE, 1993); <https://doi.org/10.1109/IJCNN.1993.716791>
- Jacobs, R. A., Jordan, M. I., Nowlan, S. J. & Hinton, G. E. Adaptive mixtures of local experts. *Neural Comput.* **3**, 79–87 (1991).
- Shazeer, N. et al. Outrageously large neural networks: the sparsely-gated mixture-of-experts layer. In *Proc. International Conference on Learning Representations (ICLR, 2017)*; <https://openreview.net/forum?id=B1ckMDqlq>
- Fedus, W., Zoph, B. & Shazeer, N. Switch transformers: scaling to trillion parameter models with simple and efficient sparsity. *J. Mach. Learn. Res.* **23**, 5232–5270 (2022).
- Raffel, C. et al. Exploring the limits of transfer learning with a unified text-to-text transformer. *J. Mach. Learn. Res.* **21**, 5485–5551 (2020).
- Du, N. et al. GLaM: efficient scaling of language models with mixture-of-experts. In *Proc. 39th International Conference on Machine Learning, Proceedings of Machine Learning Research* Vol. 162, 5547–5569 (PMLR, 2022).
- Clark, A. et al. Unified scaling laws for routed language models. In *Proc. 39th International Conference on Machine Learning* Vol. 162 (eds Chaudhuri, K. et al.) 4057–4086 (PMLR, 2022).
- Ludziejewski, J. et al. Scaling laws for fine-grained mixture of experts. In *Proc. ICLR 2024 Workshop on Mathematical and Empirical Understanding of Foundation Models* (PMLR, 2024); <https://openreview.net/forum?id=lizr8qwh7J>
- Csordás, R., Irie, K. & Schmidhuber, J. Approximating two-layer feedforward networks for efficient transformers. In *Proc. Association for Computational Linguistics: EMNLP 2023* (eds Bouamor, H. et al.) 674–692 (ACL, 2023); <https://doi.org/10.18653/v1/2023.findings-emnlp.49>
- Reuther, A. et al. AI and ML accelerator survey and trends. In *Proc. 2022 IEEE High Performance Extreme Computing Conference (HPEC)* 1–10 (IEEE, 2022); <https://doi.org/10.1109/HPEC55821.2022.9926331>
- Sebastian, A., Le Gallo, M., Khaddam-Aljameh, R. & Eleftheriou, E. Memory devices and applications for in-memory computing. *Nat. Nanotechnol.* **15**, 529–544 (2020).
- Lanza, M. et al. Memristive technologies for data storage, computation, encryption and radio-frequency communication. *Science* **376**, eabj9979 (2022).
- Mannocci, P. et al. In-memory computing with emerging memory devices: status and outlook. *APL Mach. Learn.* **1**, 010902 (2023).
- Huang, Y. et al. Memristor-based hardware accelerators for artificial intelligence. *Nat. Rev. Electr. Eng.* **1**, 286–299 (2024).
- Le Gallo, M. et al. A 64-core mixed-signal in-memory compute chip based on phase-change memory for deep neural network inference. *Nat. Electron.* **6**, 680–693 (2023).
- Ambrogio, S. et al. An analog-AI chip for energy-efficient speech recognition and transcription. *Nature* **620**, 768–775 (2023).
- Wan, W. et al. A compute-in-memory chip based on resistive random-access memory. *Nature* **608**, 504–512 (2022).
- Zhang, W. et al. Edge learning using a fully integrated neuro-inspired memristor chip. *Science* **381**, 1205–1211 (2023).
- Wen, T.-H. et al. Fusion of memristor and digital compute-in-memory processing for energy-efficient edge computing. *Science* **384**, 325–332 (2024).
- Fick, L., Skrzyniarz, S., Parikh, M., Henry, M. B. & Fick, D. Analog matrix processor for edge AI real-time video analytics. In *Proc. 2022 IEEE International Solid-State Circuits Conference (ISSCC)* Vol. 65, 260–262 (IEEE, 2022); <https://doi.org/10.1109/ISSCC42614.2022.9731773>
- Arnaud, F. et al. High density embedded PCM cell in 28 nm FDSOI technology for automotive micro-controller applications. In *Proc. 2020 IEEE International Electron Devices Meeting (IEDM)* 24.2.1–24.2.4 (IEEE, 2020); <https://doi.org/10.1109/IEDM13553.2020.9371934>
- Lee, S. et al. A 1Tb 4b/cell 64-stacked-WL 3D NAND flash memory with 12MB/s program throughput. In *Proc. 2018 IEEE International Solid-State Circuits Conference (ISSCC)* 340–342 (IEEE, 2018); <https://doi.org/10.1109/ISSCC.2018.8310323>
- Park, J.-W. et al. A 176-stacked 512Gb 3b/cell 3D-NAND flash with 10.8Gb/mm² density with a peripheral circuit under cell array architecture. In *Proc. 2021 IEEE International Solid-State Circuits Conference (ISSCC)* Vol. 64, 422–423 (IEEE, 2021); <https://doi.org/10.1109/ISSCC42613.2021.9365809>
- Lee, S.-T. & Lee, J.-H. Neuromorphic computing using NAND flash memory architecture with pulse width modulation scheme. *Front. Neurosci.* **14**, 571292 (2020).
- Bavandpour, M., Sahay, S., Mahmoodi, M. R. & Strukov, D. B. 3D-aCortex: an ultra-compact energy-efficient neurocomputing platform based on commercial 3D-NAND flash memories. *Neuromorphic Comput. Eng.* **1**, 014001 (2021).
- Shim, W. & Yu, S. Technological design of 3D NAND-based compute-in-memory architecture for GB-scale deep neural network. *IEEE Electron Device Lett.* **42**, 160–163 (2020).
- Hsieh, C.-C. et al. Chip demonstration of a high-density (43 Gb) and high-search-bandwidth (300 Gb/s) 3D NAND based in-memory search accelerator for Ternary Content Addressable Memory (TCAM) and proximity search of Hamming distance. In *Proc. 2023 IEEE Symposium on VLSI Technology and Circuits (VLSI Technology and Circuits)* 1–2 (IEEE, 2023); <https://doi.org/10.23919/VLSITechnologyandCir57934.2023.10185361>
- Huo, Q. et al. A computing-in-memory macro based on three-dimensional resistive random-access memory. *Nat. Electron.* **5**, 469–477 (2022).
- Jain, S. et al. A heterogeneous and programmable compute-in-memory accelerator architecture for analog-AI using dense 2-D mesh. *IEEE Trans. Very Large Scale Integr. VLSI Syst.* **31**, 114–127 (2023).

37. Cui, C. et al. A survey on multimodal large language models for autonomous driving. In *Proc. IEEE/CVF Winter Conference on Applications of Computer Vision (WACV) Workshops* 958–979 (IEEE, 2024).
38. Devlin, J., Chang, M.-W., Lee, K. & Toutanova, K. BERT: pre-training of deep bidirectional transformers for language understanding. In *Proc. 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies* Vol. 1, 4171–4186 (ACL, 2019); <https://doi.org/10.18653/v1/N19-1423>
39. Kim, W., Son, B. & Kim, I. ViLT: vision-and-language transformer without convolution or region supervision. In *Proc. 38th International Conference on Machine Learning* Vol. 139 (eds Meila, M. & Zhang, T.) 5583–5594 (PMLR, 2021); <https://proceedings.mlr.press/v139/kim21k.html>
40. Alayrac, J.-B. et al. Flamingo: a visual language model for few-shot learning. In *Proc. Advances in Neural Information Processing Systems* Vol. 35 (eds Koyejo, S. et al.) 23716–23736 (Curran Associates, 2022); https://proceedings.neurips.cc/paper_files/paper/2022/file/960a172bc7fbf0177ccccbb411a7d800-Paper-Conference.pdf
41. Pope, R. et al. Efficiently scaling transformer inference. In *Proc. Machine Learning and Systems* Vol. 5 (eds Song, D. et al.) 606–624 (Curran Associates, 2023); https://proceedings.mlsys.org/paper_files/paper/2023/file/c4be71ab8d24cdfb45e3d06dbfca2780-Paper-mlsys2023.pdf
42. Choquette, J., Gandhi, W., Giroux, O., Stam, N. & Krashinsky, R. NVIDIA A100 Tensor Core GPU: performance and innovation. *IEEE Micro* **41**, 29–35 (2021).
43. Radford, A. et al. Language models are unsupervised multitask learners. *Semantic Scholar* <https://api.semanticscholar.org/CorpusID:160025533> (2019).
44. Merity, S., Xiong, C., Bradbury, J. & Socher, R. Pointer sentinel mixture models. In *Proc. International Conference on Learning Representations (ICLR, 2017)*; <https://openreview.net/forum?id=Byj72udxe>
45. Vasilopoulos, A. et al. Exploiting the state dependency of conductance variations in memristive devices for accurate in-memory computing. *IEEE Trans. Electron Devices* **70**, 6279–6285 (2023).
46. Paszke, A. et al. PyTorch: an imperative style, high-performance deep learning library. In *Proc. Advances in Neural Information Processing Systems* Vol. 32, 8024–8035 (Curran Associates, 2019).
47. Reed, J. K., DeVito, Z., He, H., Ussery, A. & Ansel, J. Torch.fx: practical program capture and transformation for deep learning in Python. Preprint at <https://arxiv.org/abs/2112.08429> (2021).
48. Vaswani, A. et al. Attention is all you need. In *Proc. Advances in Neural Information Processing Systems* Vol. 30 (eds Guyon, I. et al.) (Curran Associates, 2017); https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf
49. Fisher Trace scheduling: a technique for global microcode compaction. *IEEE Trans. Comput.* **C-30**, 478–490 (1981).
50. Bernstein, D. & Rodeh, M. Global instruction scheduling for superscalar machines. In *Proc. ACM SIGPLAN 1991 Conference on Programming Language Design and Implementation PLDI '91* 241–255 (ACM, 1991); <https://doi.org/10.1145/113445.113466>
51. Joshi, V. et al. Accurate deep neural network inference using computational phase-change memory. *Nat. Commun.* **11**, 2473 (2020).
52. Kudo, T. & Richardson, J. SentencePiece: a simple and language independent subword tokenizer and detokenizer for neural text processing. In *Proc. 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations* (eds Blanco, E. & Lu, W.) 66–71 (Association for Computational Linguistics, 2018).
53. Tillet, P., Kung, H. T. & Cox, D. Triton: an intermediate language and compiler for tiled neural network computations. In *Proc. 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages* 10–19 (ACM, 2019); <https://doi.org/10.1145/3315508.3329973>
54. Le Gallo, M. et al. Using the IBM analog in-memory hardware acceleration kit for neural network training and inference. *APL Mach. Learn.* **1**, 041102 (2023).
55. Büchel, J. et al. AIHWKIT-lightning: a scalable HW-aware training toolkit for analog in-memory computing. In *Proc. Advances in Neural Information Processing Systems 2024 Workshop, Machine Learning with new Compute Paradigms* (Curran Associates, 2024); <https://openreview.net/forum?id=QNdxOgGmhR>
56. Büchel, J. et al. Gradient descent-based programming of analog in-memory computing cores. In *Proc. 2022 International Electron Devices Meeting (IEDM)* 33.1.1–33.1.4 (IEEE, 2022); <https://doi.org/10.1109/IEDM45625.2022.10019486>
57. Büchel, J. Source data for figures in ‘Efficient scaling of large language models with mixture of experts and 3D analog in-memory computing’. *Zenodo* <https://doi.org/10.5281/zenodo.14146703> (2024).
58. Büchel, J. IBM/analog-moe: code release. *Zenodo* <https://doi.org/10.5281/zenodo.14025079> (2024).
59. Büchel, J. & Vasilopolous, A. IBM/3D-CiM-LLM-Inference-Simulator: code release. *Zenodo* <https://doi.org/10.5281/zenodo.14025077> (2024).
60. Goda, A. 3D NAND technology achievements and future scaling perspectives. *IEEE Trans. Electron Devices* **67**, 1373–1381 (2020).
61. Lacaite, A. L., Spinelli, A. S. & Compagnoni, C. M. High-density solid-state storage: a long path to success. In *Proc. 2021 IEEE Latin America Electron Devices Conference (LAEDC)* 1–4 (IEEE, 2021); <https://doi.org/10.1109/LAEDC51812.2021.9437865>
62. Shoenybi, M. et al. Megatron-LM: training multi-billion parameter language models using model parallelism. Preprint at <https://arxiv.org/abs/1909.08053> (2020).

Acknowledgements

We thank G. Atwood, A. Goda and D. Mills from Micron for fruitful discussions and technical insights. We also thank R. Csordás from Stanford for valuable help in the reproduction of the Sigma-MoE results. We thank P. Diener for helping with illustrations. We also thank J. Burns from IBM and M. Helm from Micron for managerial support. We received no specific funding for this work.

Author contributions

J.B., A.V., I.B., A.R., M.L.G. and A.S. initiated the research effort. J.B. and A.V. implemented the high-level simulator. J.B. implemented the GPU kernels for training MoEs and conducted the experiments. W.A.S., J.B. and G.W.B. compared the high-level simulator to a more detailed simulator developed for 2D AIMC. B.F. and H.C. provided insights on 3D in-memory computing. H.T., V.N. and A.S. provided managerial support. J.B. and A.V. wrote the manuscript with input from all authors.

Competing interests

The authors declare no competing interests.

Additional information

Extended data is available for this paper at <https://doi.org/10.1038/s43588-024-00753-x>.

Supplementary information The online version contains supplementary material available at <https://doi.org/10.1038/s43588-024-00753-x>.

Correspondence and requests for materials should be addressed to Julian Büchel or Abu Sebastian.

Peer review information *Nature Computational Science* thanks Erika Covi, Anand Subramoney and the other, anonymous, reviewer(s) for their contribution to the peer review of this work. Primary Handling Editor: Jie Pan, in collaboration with the *Nature Computational Science* team.

Reprints and permissions information is available at www.nature.com/reprints.

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.

© The Author(s), under exclusive licence to Springer Nature America, Inc. 2025

Extended Data Table 1 | Hyperparameters underlying the results presented in Fig. 1

Parameter	Dense	MoE
Num. decoder layers	48	48
Tiles	{98,194,581,773}	{98,194,581,773}
Tiers	256	256
Tier shape	(512,512)	(512,512)
Num. DPUs	5	5
Num. MHA Units	4	4
Num. sequences (batch size)	4	4
Context length	64	64
Num. generated tokens per input sequence	8	8
Dense Model-Specific		
d_{model}	{256,512,768,1'024}	–
n_{head}	8	–
Vocab. size	50'000	–
Share embedding	False	–
MoE Model-Specific		
d_{model}	–	{256,512,768,1'024}
n_{head}	–	8
Vocab. size	–	50'000
MoE layer-frequency	–	1
Num. experts	–	{4,8,16,64,128}
top-k	–	{1,2,3,4}
Share embedding	–	False
Additional Settings		
Map strategy	Greedy (in-order)	Greedy (in-order)
Split FFN	True	True
Stack embedding	False	False

Hyperparameters used for inference time comparison between dense and MoE models.

Extended Data Table 2 | Assumed latency and energy consumption of different operations for varying model dimensions

Metric	$d_{\text{model}} = 256$	$d_{\text{model}} = 512$	$d_{\text{model}} = 768$	$d_{\text{model}} = 1024$
Latency [ns]				
MVM	100	100	100	100
MHA (seq. length 64)	4'295	16'775	37'575	66'695
MHA (seq. length 128)	17'031	66'567	149'127	264'711
MHA (seq. length 512)	270'855	1'058'823	2'372'103	4'210'695
LayerNorm	50	50	100	100
ReLU	1	1	1	1
GeLU	22	38	54	70
Add	12	20	28	36
Communication	24	24	48	48
Energy Consumption [pJ]				
MVM	10'000.0	10'000.0	10'000.0	10'000.0
MHA (seq. length 64)	23'376.37	46'743.98	70'111.60	93'483.50
MHA (seq. length 128)	93'496.90	186'971.66	280'446.41	373'925.45
MHA (seq. length 512)	1'495'907.65	2'991'525.13	4'487'142.60	5'982'764.36
LayerNorm	20.96	20.96	41.92	41.92
ReLU	0.02	0.03	0.05	0.06
GeLU	9.63	19.25	28.88	38.51
Add	9.43	18.86	28.29	37.72
Communication	12.93	25.87	38.80	51.73
Passive power [W] (5 DPUs and 4 MHA units)	0.02412	0.02412	0.02412	0.02412

The assumed latency and energy consumption of individual operations. Values are shown for relevant model dimensions and sequence lengths.

Extended Data Table 3 | TOPs and target processing unit per operation

Operation	TOPs	Target
Multi-head att.	$\begin{cases} 2 S^2 d_{\text{model}} + 4 N_{\text{dp}} + 2 N_{\text{dp}} d_{\text{model}} - S \text{ where } N_{\text{dp}} = \frac{S^2+S}{2} \text{ if causal else } S^2 \\ 2 S d_{\text{model}} + 4 N_{\text{dp}} + 2 N_{\text{dp}} d_{\text{model}} - S \text{ where } N_{\text{dp}} = S \text{ if KV-Caching} \end{cases}$	MHA Unit
LayerNorm	$10 d_{\text{model}}$	DPU
MVM (per tile)	$2 U n_{\text{rows}} n_{\text{cols}}$	Analog
ReLU	d_{model}	DPU
Communication	-	-

U is the utilization of the tile and S is the sequence length.

Extended Data Table 4 | Hyperparameters used for comparison against NVIDIA A100 GPU

Dense								
Architecture	d_{model}	Num. decoder layers		Tiles	Tiers	d_{ffn}		
GPT-2 300 mio.	1'024	24		13	256	4'096		
GPT-2 700 mio.	1'280	36		16	256	5'120		
GPT-2 1B	1'600	48		35	256	6'400		
Megatron 1B	1'536	40		20	256	6'144		
MoE								
Architecture	d_{model}	Num. decoder layers	Tiles	Tiers	d_{ffn}	Num. experts	MoE layer-frequency	top-k
MoE-base	768	24	149	256	3'072	128	2	1
MoE-large	1'024	48	392	256	4'096	128	2	1

Model architecture hyperparameters for the comparison in Fig. 4.

Extended Data Table 5 | Parameters used for the scaling laws

Model	<i>a</i>	<i>α</i>	<i>b</i>	<i>β</i>	<i>g</i>	<i>γ</i>	<i>c</i>
MoE	18.1	0.115	30.8	0.147	2.1	0.58	0.47
Dense	16.3	0.126	26.7	0.127	-	-	0.47

Fitted parameters used for the scaling law study presented in Fig. 5.

Extended Data Table 6 | Hyperparameters used for training FP-32 dense and MoE models

Parameter	MoE FP-32	MoE HW-aware	Dense FP-32	Dense HW-aware
LR	0.00025	0.00025	0.00025	0.00025
Weight decay	0.0	0.0	0.0	0.0
Seed	0	0	0	0
FP-32	True	False	True	False
Batch size	64	56	64	56
Max. gradient norm	0.25	0.25	0.25	0.25
Dropout	0.1	0.1	0.1	0.1
d_{model}	410	410	410	410
d_{ffn}	2'053	2'053	2'053	2'053
Num. decoder layers	16	16	16	16
Num. attention heads	10	10	10	10
Context length	512	512	512	512
Warmup steps	0	0	0	0
Num. experts	16	16	–	–
Expert size	128	128	–	–
top-k	4	4	–	–
MoE dropout	0.0	0.0	–	–
Router activation	Sigmoid	Sigmoid	–	–
Activation after top-k	False	False	–	–
MoE bias	False	False	–	–
Expert dropout	0.0	0.0	–	–
Weight std scale	1.0	1.0	–	–
Routing regularization	0.001	0.001	–	–
Num. MoE layers	16	16	–	–

Hyperparameters for the training procedure of the models studied in Fig. 6.

Reporting Summary

Nature Portfolio wishes to improve the reproducibility of the work that we publish. This form provides structure for consistency and transparency in reporting. For further information on Nature Portfolio policies, see our [Editorial Policies](#) and the [Editorial Policy Checklist](#).

Statistics

For all statistical analyses, confirm that the following items are present in the figure legend, table legend, main text, or Methods section.

n/a	Confirmed
☐	☒ The exact sample size (<i>n</i>) for each experimental group/condition, given as a discrete number and unit of measurement
☒	☐ A statement on whether measurements were taken from distinct samples or whether the same sample was measured repeatedly
☒	☐ The statistical test(s) used AND whether they are one- or two-sided <i>Only common tests should be described solely by name; describe more complex techniques in the Methods section.</i>
☒	☐ A description of all covariates tested
☒	☐ A description of any assumptions or corrections, such as tests of normality and adjustment for multiple comparisons
☐	☒ A full description of the statistical parameters including central tendency (e.g. means) or other basic estimates (e.g. regression coefficient) AND variation (e.g. standard deviation) or associated estimates of uncertainty (e.g. confidence intervals)
☒	☐ For null hypothesis testing, the test statistic (e.g. <i>F</i> , <i>t</i> , <i>r</i>) with confidence intervals, effect sizes, degrees of freedom and <i>P</i> value noted <i>Give P values as exact values whenever suitable.</i>
☒	☐ For Bayesian analysis, information on the choice of priors and Markov chain Monte Carlo settings
☒	☐ For hierarchical and complex designs, identification of the appropriate level for tests and full reporting of outcomes
☒	☐ Estimates of effect sizes (e.g. Cohen's <i>d</i> , Pearson's <i>r</i>), indicating how they were calculated

Our web collection on [statistics for biologists](#) contains articles on many of the points above.

Software and code

Policy information about [availability of computer code](#)

Data collection	Software used in this study that is not common software (such as Python and basic packages building on Python): - Code to run experiments using our simulated accelerator: https://github.com/IBM/3D-CiM-LLM-Inference-Simulator - Code to train MoE layers in a HW-aware fashion: https://github.com/IBM/analog-moe
Data analysis	For data analysis and plotting, Python, Matplotlib and Inkscape were used.

For manuscripts utilizing custom algorithms or software that are central to the research but not yet described in published literature, software must be made available to editors and reviewers. We strongly encourage code deposition in a community repository (e.g. GitHub). See the Nature Portfolio [guidelines for submitting code & software](#) for further information.

Data

Policy information about [availability of data](#)

All manuscripts must include a [data availability statement](#). This statement should provide the following information, where applicable:

- Accession codes, unique identifiers, or web links for publicly available datasets
- A description of any restrictions on data availability
- For clinical datasets or third party data, please ensure that the statement adheres to our [policy](#)

The underlying data of all figures is supplied in the supplementary material. Instructions on how to access the data needed to re-generate the results are present in the accompanying GitHub repositories. The WikiText-103 dataset is publicly available at <https://huggingface.co/datasets/Salesforce/wikitext>.

Human research participants

Policy information about [studies involving human research participants and Sex and Gender in Research](#).

Reporting on sex and gender	N/A
Population characteristics	N/A
Recruitment	N/A
Ethics oversight	N/A

Note that full information on the approval of the study protocol must also be provided in the manuscript.

Field-specific reporting

Please select the one below that is the best fit for your research. If you are not sure, read the appropriate sections before making your selection.

☒ Life sciences ☐ Behavioural & social sciences ☐ Ecological, evolutionary & environmental sciences

For a reference copy of the document with all sections, see nature.com/documents/nr-reporting-summary-flat.pdf

Life sciences study design

All studies must disclose on these points even when the disclosure is negative.

Sample size	N=5 for most experiments.
Data exclusions	We did not exclude any data.
Replication	All experimental results can be replicated using the submitted code.
Randomization	Randomization is performed on the experiment level.
Blinding	Blinding does not apply to our study.

Reporting for specific materials, systems and methods

We require information from authors about some types of materials, experimental systems and methods used in many studies. Here, indicate whether each material, system or method listed is relevant to your study. If you are not sure if a list item applies to your research, read the appropriate section before selecting a response.

Materials & experimental systems

Methods

n/a	Involved in the study
☒	☐ Antibodies
☒	☐ Eukaryotic cell lines
☒	☐ Palaeontology and archaeology
☒	☐ Animals and other organisms
☒	☐ Clinical data
☒	☐ Dual use research of concern

n/a	Involved in the study
☒	☐ ChIP-seq
☒	☐ Flow cytometry
☒	☐ MRI-based neuroimaging